ADR-001: Usar Domain Model como patrón de lógica de negocio

- **Estado:** Aceptada
- **Contexto del curso:** Componentes de lógica de negocio (FarmaExpress, Equipo 10)

Contexto

El BRD de FarmaExpress define 16 reglas de negocio (BR-01 a BR-16) que se combinan en una misma operación de venta (existencia, caducidad, lote retirado, requisito especial, autorización). El BPMN tiene 7 compuertas con caminos de rechazo. Los lotes tienen estados (disponible, caducado, retirado). Hay reglas por definir (BR-12, políticas de requisitos especiales) y requisitos de trazabilidad y consistencia (RNF-02, RNF-03, RNF-05). Además, el negocio exige eventos (BR-09: avisar a Compras) y el glosario tiene 43 términos.

Decisión

Usar **Domain Model** para el lado de escritura (commands). Las consultas y reportes se resuelven con lecturas simples del lado Query, sin pasar por el modelo.

Alternativas consideradas

Alternativa	Por qué se descartó
Transaction Script	Se descartó porque, aunque es sencillo para aplicaciones pequeñas y CRUDs básicos, en el caso de FarmaExpress provocaría una duplicación masiva de código (Code Duplication) y una alta complejidad al intentar gestionar las reglas de validación de medicamentos, el control estricto de lotes, la caducidad y los flujos complejos de sustitución. Al crecer el sistema, los scripts se volverían difíciles de mantener y altamente propensos a errores de consistencia en el inventario.
Table Module	Se descartó porque está demasiado acoplado al modelo de datos relacional y no permite representar de forma natural los comportamientos complejos del dominio farmacéutico (como los agregados de Venta, las reglas de vendibilidad por lote o las políticas de alertas de inventario mínimo). Introduciría un acoplamiento indeseado entre la estructura de las tablas de la base de datos y la lógica del negocio.

Consecuencias

- (+) Separación estricta y limpia entre el dominio del negocio, la capa de aplicación y los detalles de infraestructura, facilitando la comprensión y evolución del código.
- (+) Protección robusta de las invariantes críticas del negocio (como la verificación de caducidad de lotes, el rechazo de pagos antes de tocar el inventario y la validación de recetas o requisitos especiales).
- (+) Lenguaje ubicuo reflejado de manera directa en el diseño y en el código, lo que alinea las necesidades del BRD y las historias de usuario con los componentes técnicos implementados.

- (+) Facilidad para realizar pruebas unitarias sobre las reglas de negocio de forma aislada, sin necesidad de levantar bases de datos ni servicios externos.
- (-) Curva de aprendizaje y complejidad inicial más alta en comparación con un enfoque de scripts simples o programación lineal tradicional.
- (-) Mayor cantidad de clases y interfaces, lo que requiere una mayor disciplina por parte del equipo para mantener la consistencia estructural.
- (-) Mayor inversión de tiempo en el diseño inicial antes de comenzar con la codificación de las características funcionales.

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-01, OB-02, OB-03, OB-04, OB-05	El modelo protege las reglas que hacen cumplir estos objetivos
Necesidades	N-01, N-02, N-03, N-04, N-06, N-08, N-11	Existencia, caducados, lotes retirados, requisitos especiales, existencia baja, trazabilidad por lote y sustitución
Reglas de negocio	BR-01 a BR-05, BR-07 a BR-15	Cada regla vive en el objeto dueño, no en un script
Requerimientos funcionales	RF-02 a RF-07, RF-09 a RF-12, RF-16	Comportamiento que modelan los aggregates
Requerimientos no funcionales	RNF-02, RNF-03, RNF-05	Invariantes de consistencia y trazabilidad dentro del aggregate
User stories	US-01, US-02, US-03, US-04, US-07, US-08, US-14, US-17 (US-11 como consecuencia de CU-04)	Base de los 5 casos de uso
Casos de uso	CU-01 a CU-05	Todos usan el Domain Model
Diagramas	`Mapa de dominio.png`	Representa el modelo
ADR relacionados	ADR-002, ADR-003, ADR-006	Aggregates, Service Layer y reglas de vendibilidad

ADR-002: Aggregates que se referencian por identificador

- **Estado:** Aceptada

Contexto

Se necesitan límites de consistencia claros. FarmaExpress opera con varias sucursales y, según el glosario, la existencia es la cantidad de un producto **en una sucursal específica**. El nivel mínimo (BR-09) se evalúa sobre el total del producto en esa sucursal, no por lote. Una venta toca varios productos, pero siempre de una sola sucursal.

Decisión

Cinco aggregates: `Venta` (con `LineaDeVenta`), `InventarioDeProducto` (con `Lote`), `SolicitudDeSustitucion`, `RecepcionDeMercancia` (con `LineaDeRecepcion`) y `Producto` (catálogo). Entre aggregates solo se guarda el **identificador** (productoId, sucursalId, lote, ventaId), nunca el objeto.

`InventarioDeProducto` se identifica por **producto y sucursal** y agrupa los lotes de ese producto en esa sucursal, porque la regla de nivel mínimo cruza lotes. `Venta` y `RecepcionDeMercancia` llevan la `SucursalId` donde ocurren.

Alternativas consideradas

Alternativa	Por qué se descartó
Un solo aggregate `Inventario` con todos los productos	Contención: todas las ventas competirían por el mismo objeto.
Un aggregate por producto con todas sus sucursales	Una venta en una sucursal bloquearía a las demás sin necesidad.
Un aggregate por lote	El nivel mínimo no podría evaluarse dentro de una sola transacción.
Venta con referencia directa al inventario	Acopla dos aggregates y rompe la regla de consistencia por aggregate.

Consecuencias

- (+) Aggregates pequeños y fáciles de cargar y guardar; las ventas de sucursales distintas no compiten entre sí.
- (-) Las operaciones que tocan varios aggregates necesitan una unidad de trabajo (se puede ver en ADR-004).

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-01, OB-07	Control de inventario y visibilidad por sucursal
Necesidades	N-01, N-05, N-08	Existencia, visibilidad entre sucursales y trazabilidad por lote
Reglas de negocio	BR-01, BR-02, BR-03, BR-06, BR-07, BR-09, BR-15	Se protegen dentro de `InventarioDeProducto`, `Venta` y `RecepcionDeMercancia`
Requerimientos funcionales	RF-02, RF-07, RF-09, RF-10, RF-11	Operaciones sobre los aggregates
Requerimientos no funcionales	RNF-02, RNF-05	Integridad y consistencia de producto, lote, sucursal y caducidad
Datos	D-03 (lote), D-04 (caducidad), D-05 (sucursal), D-06 (nivel mínimo), D-07 (estado del lote)	Atributos de `InventarioDeProducto` y `Lote`
User stories	US-01, US-02, US-14, US-17 (soporta US-09 y US-11)	Casos de uso que cargan y guardan estos aggregates
Casos de uso	CU-01, CU-03, CU-04, CU-05	Cada uno toca uno o varios aggregates
Diagramas	`Mapa de dominio.png`	Aggregates y referencias por id
ADR relacionados	ADR-004, ADR-007	Unidad de trabajo para operaciones entre aggregates y alcance por sucursal

ADR-003: Service Layer con un servicio por caso de uso y CQRS

- **Estado:** Aceptada

Contexto

La capa de aplicación debe orquestar sin contener reglas de negocio. Un servicio único (`ServicioDeVentas`) con 8 métodos y 6 dependencias violaría SRP e ISP.

Decisión

- Un Application Service por caso de uso, con un solo método `ejecutar(comando)`: `IniciarVenta`, `RegistrarProductoEnVenta`, `ResolverValidacionEspecial`, `SolicitarSustitucion`, `ValidarSustitucion`, `RegistrarDecisionDelCliente`, `CompletarVenta`, `RegistrarRecepcionDeMercancia`.
- **CQRS básico:** los *commands* modifican y pasan por el dominio; los *queries* (`ConsultarExistenciaDeProducto`, `ConsultarValidacionesPendientes`, `ConsultarAlternativasDisponibles`, `ConsultarResumenDeVenta`, `ConsultarDisponibilidadPorSucursal`) solo leen desde lectores de vista y devuelven DTOs, sin cargar aggregates.
- Los commands devuelven `ResultadoDeOperacion`. Un rechazo de negocio (sin existencia, lote caducado, autorización denegada, pago rechazado) es un resultado, no una excepción.
- Los servicios no contienen reglas: cargan, delegan al dominio, guardan y confirman.

Alternativas consideradas

Alternativa	Por qué se descartó
Un `ServicioDeVentas` con todos los métodos	Clase que hace todo; constructor con demasiadas dependencias.
CQRS completo con almacenes separados	Fuera de alcance: el sistema es ficticio y no hay bases de datos.

Consecuencias

- (+) Clases pequeñas, con dependencias mínimas y fáciles de probar.
- (-) Más clases (8 commands y 5 queries).

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-02, OB-07, OB-08, OB-10	Venta correcta, visibilidad, apoyo a resurtido e información para decisiones

Tipo	Referencias	Relación con la decisión
Necesidades	N-01, N-04, N-05, N-10, N-11	Cubiertas por commands y queries
Reglas de negocio	BR-04, BR-05, BR-08, BR-16	Los servicios las disparan, pero las reglas viven en el dominio
Requerimientos funcionales	RF-01, RF-02, RF-05, RF-06, RF-07, RF-08, RF-09, RF-16	Un command o query por capacidad
Requerimientos no funcionales	RNF-01, RNF-07	La verificación de permisos por rol se hace en esta capa, no en el dominio
User stories (commands)	US-01, US-03, US-07, US-08, US-14, US-17	`RegistrarProductoEnVenta`, `ResolverValidacionEspecial`, `SolicitarSustitucion`, `ValidarSustitucion`, `RegistrarDecisionDelCliente`, `RegistrarRecepcion` `CompletarVenta`
User stories (queries)	US-02, US-03, US-13	`ConsultarExistenciaDeProducto`, `ConsultarValidacionesPendientes`, `ConsultarDisponibilidadPorSucursal`
Casos de uso	CU-01 a CU-05	Un servicio por caso de uso (CU-03 tiene tres)
Diagramas	`Capa de aplicacion.png`, `secuencia 1.png`, `secuencia 2.png`	Estructura y orquestación
ADR relacionados	ADR-001, ADR-004, ADR-005	Dominio, unidad de trabajo y repositorios que usan los servicios

ADR-004: Unit of Work y orden de cobro en CompletarVenta

- **Estado:** Aceptada

Contexto

Algunos casos de uso modifican varios aggregates y deben quedar completos o no quedar (RNF-02, RNF-05). Además, completar una venta involucra un cobro externo que puede ser rechazado.

Decisión

1. Se confirman juntos, en una `UnidadDeTrabajo` todo o nada:
   - `CompletarVenta`: Venta + inventarios afectados.
   - `RegistrarRecepcionDeMercancia`: Recepción + inventarios.
   - `RegistrarDecisionDelCliente` (cuando acepta): Solicitud + Venta.
2. Orden de `CompletarVenta`: verificar que la venta puede cobrarse y que cada línea es vendible → cobrar → descontar y confirmar. Si el pago es rechazado, solo se confirma el rechazo de la venta y el inventario no se toca.
3. Los eventos de dominio (`VentaCompletada`, `ExistenciaBajaDetectada`, etc.) se publican después de confirmar.

Alternativas consideradas

Alternativa	Por qué se descartó
Descontar en memoria antes de cobrar y deshacer si falla el pago	Más complejo: obliga a descartar cambios y abrir una segunda unidad de trabajo solo para registrar el rechazo.
Consistencia eventual entre Venta e inventario	RNF-05 exige disponibilidad actualizada después de cada operación.

Consecuencias

- (+) El inventario nunca queda descontado por una venta no cobrada.
- (-) Si la confirmación falla después de un cobro aceptado se necesitaría una compensación (reembolso); queda fuera del alcance del ejercicio.
- (-) Dos cajas de la misma sucursal pueden vender la última unidad al mismo tiempo; `descontar` vuelve a validar y la unidad de trabajo usaría control de versión optimista.

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-01, OB-02, OB-03, OB-04, OB-05	Inventario correcto y sin ventas de productos no vendibles

Tipo	Referencias	Relación con la decisión
Necesidades	N-01, N-02, N-03, N-06, N-08, N-10	Verificar antes de cobrar, descontar y alertar
Reglas de negocio	BR-01, BR-02, BR-03, BR-06, BR-07, BR-08, BR-09, BR-10, BR-15	Se verifican antes del cobro y se aplican al confirmar
Requerimientos funcionales	RF-02, RF-07, RF-09, RF-10, RF-11, RF-12, RF-16	Operaciones que se guardan juntas
Requerimientos no funcionales	RNF-02, RNF-03, RNF-05	Todo o nada, con evidencia de entradas y salidas
User stories	US-17, US-14, US-07, US-08 (y US-11 como evento posterior)	CU-04, CU-05 y CU-03
Casos de uso	CU-04 (principal), CU-05, CU-03	Operaciones con varios aggregates
Diagramas	`Secuencia 1.png` (principal), `Secuencia 2.png`, `Capa de aplicacion.png`	Muestran el orden y la confirmación conjunta
ADR relacionados	ADR-002, ADR-003, ADR-005	Aggregates, servicios y puerto `UnidadDeTrabajo`

ADR-005: Repositorios y puertos

- **Estado:** Aceptada

Contexto

El dominio debe recuperar y guardar información sin conocer tecnologías. Además, la trazabilidad (RNF-03) y la auditabilidad (RNF-08) exigen conservar el historial.

Decisión

- Repositorios con nombres de colección del negocio y máximo dos métodos: `Ventas`, `InventariosDeProductos`, `Productos`, `SolicitudesDeSustitucion`, `Recepciones`. Ninguno permite eliminar. El inventario se obtiene por producto y sucursal: `obtenerDe(producto, sucursal)`.
- Puertos de una sola responsabilidad: `UnidadDeTrabajo` (confirmar, descartar), `ProcesadorDePagos` (cobrar), `RelojDelNegocio` (hoy), `PublicadorDeEventos` (publicar).
- Lectores de solo consulta para el lado Query: `LectorDeVentas`, `LectorDeInventario`, `LectorDeCatalogo`.
- Las implementaciones son ficticias (en memoria); no hay base de datos.

Alternativas consideradas

Alternativa	Por qué se descartó
`Repositorio<T>` genérico con CRUD completo	Interfaz grande (ISP) y expone eliminar, contrario a la trazabilidad.
Acceso directo a almacenamiento desde los servicios	Acopla la lógica a la tecnología (DIP).

Consecuencias

- (+) Interfaces mínimas y sustituibles; el dominio no depende de infraestructura.
- (-) Más interfaces que implementar.

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-01, OB-02, OB-03	Datos consistentes y trazables
Necesidades	N-08, N-09	Trazabilidad por lote y control de cambios
Reglas de negocio	BR-02 (requiere la fecha actual), BR-14	Sin eliminar ni ajustar directamente; la fecha llega por `RelojDelNegocio`
Requerimientos funcionales	RF-15	No existe una operación de ajuste o borrado directo

Tipo	Referencias	Relación con la decisión
Requerimientos no funcionales	RNF-02, RNF-03, RNF-08	Historial conservado y auditable
Casos de uso	CU-01 a CU-05	Todos usan estos repositorios y puertos
Diagramas	`Capa de aplicacion.png`	Repositorios, lectores y puertos
ADR relacionados	ADR-003, ADR-004, ADR-006	Servicios, unidad de trabajo y reglas que reciben la fecha

ADR-006: Reglas de vendibilidad como estrategias intercambiables

- **Estado:** Aceptada

Contexto

El BRD deja reglas sin definir: el periodo de proximidad a caducidad (BR-12) y las políticas de ciertos requisitos especiales, que deben validarse con el stakeholder. Se espera que cambien y no se quiere modificar `Lote` cada vez (principio abierto/cerrado).

Decisión

Definir la interfaz `ReglaDeVendibilidad` (`cumple(lote, hoy)` y `motivo()`) con las implementaciones `LoteNoCaducado` y `LoteNoRetirado`. `Lote.esVendible(hoy)` evalúa la lista de reglas. Una regla nueva (por ejemplo, proximidad a caducar) se agrega como una clase nueva. El dominio recibe la fecha (`hoy`) como parámetro y no la consulta al sistema.

Alternativas consideradas

Alternativa	Por qué se descartó
`if` dentro de `Lote.esVendible`	Cada regla nueva obliga a modificar la clase existente.

Consecuencias

- (+) Se extiende sin modificar código existente y las reglas se prueban por separado.
- (-) Una capa más de indirección para solo dos reglas iniciales.

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-03, OB-04, OB-06	Caducados, retirados y próximos a caducar
Necesidades	N-02, N-03, N-07	Bloquear caducados y retirados; identificar próximos a caducar
Reglas de negocio	BR-02, BR-03, BR-11, BR-12 (pendiente)	`LoteNoCaducado` y `LoteNoRetirado` hoy; la proximidad a caducar sería una regla nueva
Requerimientos funcionales	RF-03, RF-04, RF-13	Los dos primeros ya están; RF-13 se agregaría como regla
User stories	US-04 (usada); US-05 y US-06 (fuera de los 5 casos, pero encajan sin cambiar `Lote`)	Ampliaciones futuras

Tipo	Referencias	Relación con la decisión
Casos de uso	CU-01, CU-03 (la alternativa sigue vendible), CU-04 (verificar antes de cobrar)	Usan `esVendible(hoy)`
Diagramas	`Mapa de dominio.png`	`ReglaDeVendibilidad` y sus implementaciones
ADR relacionados	ADR-001, ADR-002, ADR-005	Domain Model, aggregate `InventarioDeProducto` y fecha por `RelojDelNegocio`

ADR-007: Alcance: varias sucursales y composición del medicamento

- **Estado:** Aceptada

Contexto

El BRD define que FarmaExpress opera con varias sucursales (OB-07, RF-08, BR-16, US-13, dato D-05). La composición del medicamento no es un dato del sistema: el personal de farmacia la explica al cliente de palabra al evaluar la sustitución. Finalmente, el descuento del inventario después de cada venta lo hace el sistema automáticamente (US-17); el personal de almacén solo verifica la recepción, los niveles y cuándo comprar.

Decisión

- Existe un `InventarioDeProducto` por **producto y sucursal**; sus lotes pertenecen a esa sucursal. El nivel mínimo se evalúa por producto y sucursal.
- `Venta` y `RecepcionDeMercancia` llevan `SucursalId`. Una venta solo descuenta del inventario de su sucursal y una recepción solo ingresa mercancía en la suya.
- La visibilidad entre sucursales (US-13, BR-16) se resuelve del lado Query con `ConsultarDisponibilidadPorSucursal`, que solo lee y no modifica nada.
- Los traslados de mercancía entre sucursales quedan fuera del alcance (no hay user story que los pida).
- No se modela la composición del producto.
- La decisión del cliente sobre la sustitución la registra el cajero (el cliente no opera el sistema).
- En US-17 el actor es el **Sistema**.

Alternativas consideradas

Alternativa	Por qué se descartó
Inventario global sin sucursal	El BRD y el glosario piden existencia y disponibilidad por sucursal.
Un aggregate con todas las sucursales de un producto	Contención entre sucursales (se puede ver en ADR-002).

Consecuencias

- (+) El modelo es coherente con el BRD y el glosario.
- (+) Las ventas de sucursales distintas no se bloquean entre sí.
- (-) `ExistenciaBajaDetectada` y las alertas para Compras son por producto y sucursal.
- (-) La consulta entre sucursales lee una vista de lectura y su dato puede estar ligeramente atrasado respecto al inventario real.

Trazabilidad

Tipo	Referencias	Relación con la decisión
Objetivos de negocio	OB-07, OB-01	Visibilidad entre sucursales y control de inventario
Necesidades	N-05, N-11	Visibilidad entre sucursales y atención ante productos no disponibles
Reglas de negocio	BR-06, BR-07, BR-08, BR-09, BR-16	Inventario por sucursal, descuento automático, sustitución validada por farmacia
Requerimientos funcionales	RF-07, RF-08, RF-16	Descuento tras la venta, consulta por sucursal y sustituciones
User stories	US-13 (consulta entre sucursales), US-17 (actor Sistema), US-07 y US-08 (sustitución sin composición)	Decisiones de alcance
Casos de uso	CU-01, CU-03, CU-04, CU-05	Todos operan sobre la sucursal de la venta o de la recepción
Diagramas	`Secuencia 1.png`, `Secuencia 2.png`, `Capa de aplicacion.png`, `Mapa de dominio.png`	`SucursalId`, `ConsultarDisponibilidadPorSucursal` y el flujo de sustitución
ADR relacionados	ADR-002, ADR-003	Identidad del aggregate y query de disponibilidad